

On This Page >

wireguard

Wireguard-based VPN mesh network with automatic IPv6 address allocation

System

Network

This service provides a WireGuard-based VPN mesh network with automatic IPv6 address allocation and routing between clan machines.

Overview

The WireGuard service creates a secure mesh network between clan machines using two roles:

- **Controllers:** Machines with public endpoints that act as connection points and routers
- **Peers:** Machines that connect through controllers to access the network

Requirements

- Controllers must have a publicly accessible endpoint (domain name or static IP)
- Peers must be in networks where UDP traffic is not blocked (uses port 51820 by default, configurable)

Features

- Automatic IPv6 address allocation using ULA (Unique Local Address) prefixes
- Full mesh connectivity between all machines
- Automatic key generation and distribution

- IPv6 forwarding on controllers for inter-peer communication
- Support for multiple controllers for redundancy

Network Architecture

IPv6 Address Allocation

- Base network: `/40` ULA prefix (deterministically generated from instance name)
- Controllers: Each gets a `/56` subnet from the base `/40`
- Peers: Each gets a unique 64-bit host suffix that is used in ALL controller subnets

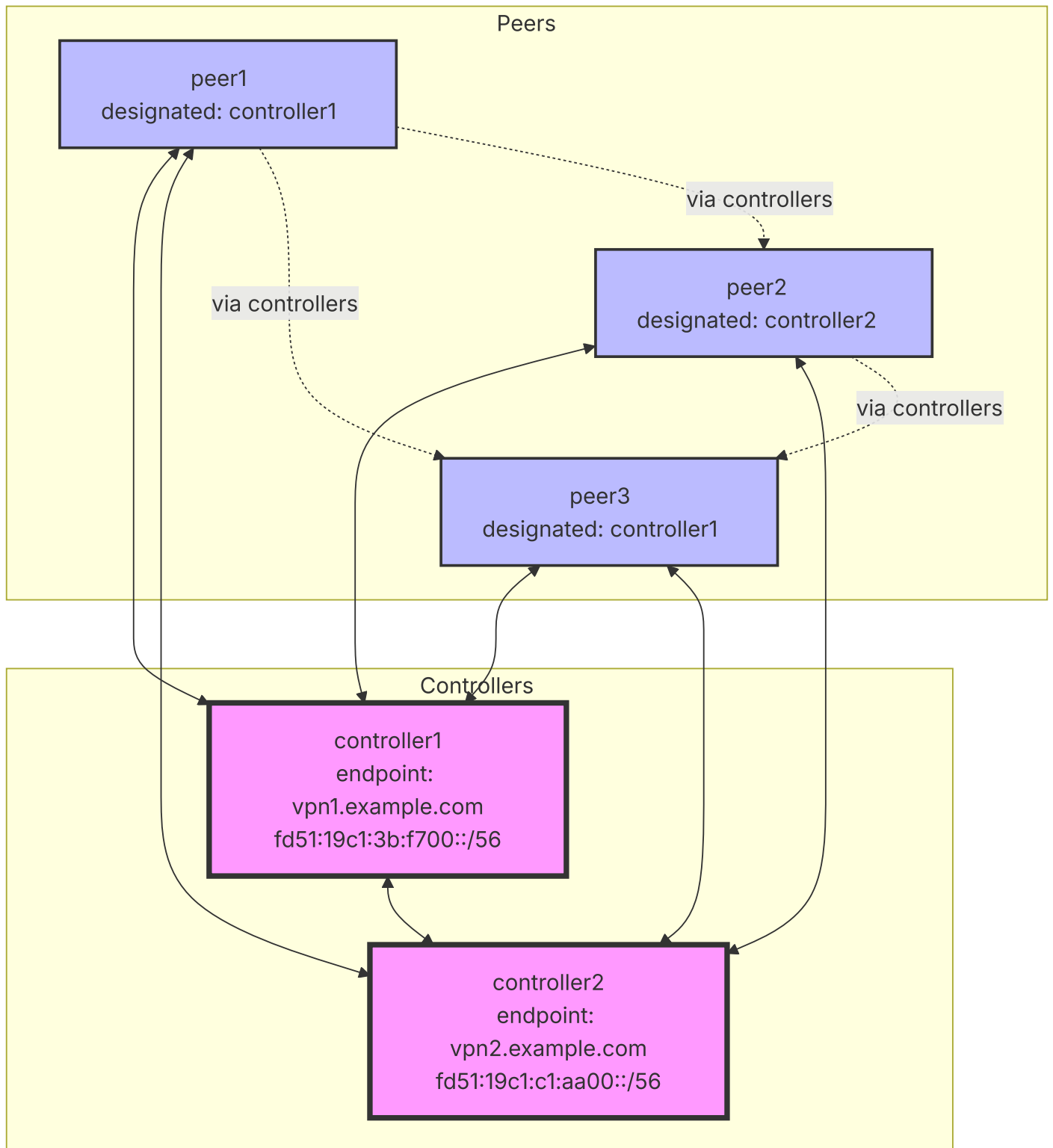
Addressing Design

- Each peer generates a unique host suffix (e.g., `:8750:a09b:0:1`)
- This suffix is appended to each controller's `/56` prefix to create unique addresses
- Example: peer1 with suffix `:8750:a09b:0:1` gets:
 - `fd51:19c1:3b:f700:8750:a09b:0:1` in controller1's subnet
 - `fd51:19c1:c1:aa00:8750:a09b:0:1` in controller2's subnet
- Controllers allow each peer's `/96` subnet for routing flexibility

Connectivity

- Peers use a single WireGuard interface with multiple IPs (one per controller subnet)
- Controllers connect to ALL other controllers and ALL peers on a single interface
- Controllers have IPv6 forwarding enabled to route traffic between peers
- All traffic between peers flows through controllers
- Symmetric routing is maintained as each peer has consistent IPs across all controllers

Example Network Topology



Configuration

Basic Setup with Single Controller

```
1 # In your clan.nix
2 {
3     instances = {
4         wireguard = {
5             module.name = "wireguard";
6             module.input = "clan-core";
7             roles.controller = {
8                 machines.server1 = {};
9                 settings = {
10                     # Public endpoint where this controller can be reached
11                     endpoint = "vpn.example.com";
12                     # Optional: Change the UDP port (default: 51820)
13                     port = 51820;
14                 };
15             };
16             roles.peer = {
17                 # No configuration needed if only one controller exists
18                 machines.laptop1 = {};
19             };
20         };
21     };
22 }
```

Multiple Controllers Setup

```
1 {
2     instances = {
3         wireguard = {
4             module.name = "wireguard";
5             module.input = "clan-core";
6             roles.controller.machines = {
7                 server1.settings.endpoint = "vpn1.example.com";
8                 server2.settings.endpoint = "vpn2.example.com";
9                 server3.settings.endpoint = "vpn3.example.com";
10            };
11            roles.peer.machines.laptop1 = {
12                # Must specify which controller subnet is exposed as the default
13                settings.controller = "server1";
14            };
15        };
16    }
```

```
17 }
```

Advanced Options

Automatic Hostname Resolution

The WireGuard service automatically adds entries to `/etc/hosts` for all machines in the network. Each machine is accessible via its hostname in the format `<machine-name>.<instance-name>`.

For example, with an instance named `vpn`:

- `server1.vpn` - resolves to server1's IPv6 address
- `laptop1.vpn` - resolves to laptop1's IPv6 address

This allows machines to communicate using hostnames instead of IPv6 addresses:

```
1 # Ping another machine by hostname
2 ping6 server1.vpn
3
4 # SSH to another machine
5 ssh user@laptop1.vpn
```

Troubleshooting

Check WireGuard Status

```
sudo wg show
```

Verify IP Addresses

```
ip addr show dev <instance-name>
```

Check Routing

```
ip -6 route show dev <instance-name>
```

Interface Fails to Start: "Address already in use"

If you see this error in your logs:

```
wireguard: Could not bring up interface, ignoring: Address already in use
```

This means the configured port (default: 51820) is already in use by another service or WireGuard instance. Solutions:

1. Check for conflicting WireGuard instances:

```
sudo wg show
sudo ss -ulnp | grep 51820
```

2. Use a different port:

```
1 services.wireguard.myinstance = {
2   roles.controller = {
3     endpoint = "vpn.example.com";
4     port = 51821; # Use a different port
5   };
6 };
```

3. Ensure unique ports across multiple instances: If you have multiple WireGuard instances on the same machine, each must use a different port.

Key Management

Keys are automatically generated and stored in the clan vars system. To regenerate keys:

```
1 # Regenerate keys for a specific machine and instance
2 clan vars generate --service wireguard-keys-<instance-name> --regenerate
3
4 # Apply the new keys
5 clan machines update <machine-name>
```

Security Considerations

- All traffic is encrypted using WireGuard's modern cryptography
 - Private keys never leave the machines they're generated on
 - Public keys are distributed through the clan vars system
 - Controllers must have publicly accessible endpoints
 - Firewall rules are automatically configured for the WireGuard ports
-

Roles

The wireguard service has the following roles:

- controller
- peer

Options for the `controller` role

domain

Domain suffix to use for hostnames in /etc/hosts. Defaults to the instance name.

Type: `null or string`

Default:

instanceName

Declared in: [clanServices/wireguard/default.nix](#)

endpoint

Endpoint where the controller can be reached

Type: `string`

Example

```
"vpn.clan.lol"
```

Declared in: [clanServices/wireguard/default.nix](#)

port

Port for the wireguard interface

Type: `signed integer`

Default:

51820

Example

51820

Declared in: [clanServices/wireguard/default.nix](#)

Options for the `peer` role

controller

Machinename of the controller to attach to

Type: `string`

❖ Example

```
"controller1"
```

Declared in: [clanServices/wireguard/default.nix](#)

domain

Domain suffix to use for hostnames in /etc/hosts. Defaults to the instance name.

Type: `null or string`

Default:

`instanceName`

Declared in: [clanServices/wireguard/default.nix](#)

port

Port for the wireguard interface

Type: `signed integer`

Default:

51820

❖ Example

51820

Declared in: [clanServices/wireguard/default.nix](#)

← Previous
wifi

Next →
yggdrasil

Need help? Reach out to [our community](#).